



CLICKGEO

CURSOS DE GEOTECNOLOGIAS

Introdução ao R Part 2

Função

Funções em R são unidades fundamentais de código reutilizável projetadas para executar uma tarefa específica. A definição de uma função é feita utilizando a palavra-chave `function`, seguida por parênteses contendo quaisquer argumentos necessários. As funções em R podem retornar valores implicitamente, ou seja, o último valor avaliado na função é retornado automaticamente.

Como exemplo prático, vamos definir uma função chamada `imc`, que calcula o Índice de Massa Corporal (IMC). O IMC é uma medida que utiliza o peso (em quilogramas) e a altura (em metros) de uma pessoa para calcular seu índice corporal. A função `imc` recebe dois parâmetros: `peso` e `altura`, e retorna o IMC calculado, arredondado para duas casas decimais.

```
imc <- function(peso, altura) {  
  round(peso / (altura^2), 2)  
}
```

```
# Exemplo de uso da função  
print(imc(50, 1.4))
```

```
## [1] 25.51
```

Avançando na versão da função do IMC, além de calcular o Índice de Massa Corporal (IMC), também fornecemos uma avaliação sobre a categoria de peso de acordo com o IMC calculado. Isso é realizado por meio de uma série de instruções `if`, `else if` e `else`, que classificam o IMC em diferentes categorias de peso.

```
imc <- function(peso, altura) {  
  indice <- round(peso / (altura^2), 2)  
  classificacao <- ""  
  
  if (indice <= 18.5) {  
    classificacao <- "Abaixo do Peso"  
  } else if (indice <= 24.9) {  
    classificacao <- "Peso Normal"  
  } else if (indice <= 29.99) {  
    classificacao <- "Sobrepeso"  
  } else if (indice <= 34.99) {  
    classificacao <- "Obesidade Grau I"  
  }
```

```

    } else if (indice <= 39.99) {
      classificacao <- "Obesidade Grau II"
    } else {
      classificacao <- "Obesidade Grau III"
    }

    return(list(IMC = indice, Categoria = classificacao))
}

# Exemplo de uso da função
resultado <- imc(50, 1.4)
print(paste("IMC:", resultado$IMC, ", Categoria:", resultado$Categoria))

```

```
## [1] "IMC: 25.51 , Categoria: Sobrepeso"
```

Função anônimas em R

Funções anônimas em R, equivalentes às expressões lambda, são criadas com a função `function` sem a necessidade de nomeá-las. São úteis para operações simples que podem ser expressas em uma única expressão.

```

# Exemplo de função anônima em R
(function(x) x^2)(2)

```

```
## [1] 4
```

```
(function(a, b) a + b)(3, 5)
```

```
## [1] 8
```

```

# Exemplo de aplicação de função anônima
pessoas <- list(
  list(nome = "Ana", idade = 20),
  list(nome = "Bruno", idade = 17),
  list(nome = "Carlos", idade = 23),
  list(nome = "Diana", idade = 15)
)

```

```

# Filtrar pessoas com mais de 18 anos e converter nomes para maiúsculas
nomes_maiusculos <- lapply(Filter(function(p) p$idade > 18, pessoas), function(p) toupper(p$nome))
print(nomes_maiusculos)

```

```

## [[1]]
## [1] "ANA"
##
## [[2]]
## [1] "CARLOS"

```

Estruturas de Teste e Tratamento de Erros

As estruturas de teste e tratamento de erros em R são essenciais para garantir a robustez e a confiabilidade dos programas. Elas permitem identificar e gerenciar situações excepcionais, evitando interrupções abruptas no código. As principais estruturas para isso são o uso de funções como `stop`, `warning`, `try`, `tryCatch`, além das funções de teste em pacotes como `testthat`.

Teste com Condições

No R podemos realizar verificações e parar a execução se uma condição não for atendida usando a função `stop`.

```
testar_divisao <- function(x) {  
  if (x == 0) {  
    stop("Divisor não pode ser zero.")  
  }  
  return(10 / x)  
}  
  
# Testando a função  
#testar_divisao(0)
```

Try e TryCatch

Outra função é o `try` e `tryCatch` para tratamento de erros. Com `try`, se um erro ocorre, o código continua a ser executado. Com `tryCatch`, podemos lidar especificamente com erros.

```
# Uso de try  
resultado <- try(10 + 'a', silent = FALSE)
```

```
## Error in 10 + "a" : non-numeric argument to binary operator
```

```
# Uso de tryCatch  
tryCatch({  
  resultado <- 10 + 'a'  
}, warning = function(w) {  
  print("Um aviso ocorreu.")  
}, error = function(e) {  
  print("Um erro ocorreu.")  
}, finally = {  
  print("Execução finalizada.")  
})
```

```
## [1] "Um erro ocorreu."  
## [1] "Execução finalizada."
```

Testes Unitários com Testthat

O pacote `testthat` é comumente usado em R para escrever testes unitários. Ele oferece uma abordagem similar ao `unittest` do Python.

```
#install.packages("testthat")
```

```
#Importando biblioteca  
library(testthat)
```

```
# Definindo testes
test_that("Testando soma", {
  expect_equal(1 + 2, 3)
  #expect_equal(1 + 2, 4)
})
```

```
## Test passed
```

Importação de Bibliotecas

No ecossistema R, existem diversas bibliotecas e pacotes importantes que atendem a uma ampla gama de necessidades, desde análise de dados até visualização e modelagem estatística.

Instalação de Pacotes

Para instalar pacotes em R, usamos a função `install.packages()`.

```
# Instalando um pacote em R
#install.packages("ggplot2")
```

Tidyverse (inclui dplyr e ggplot2) <https://www.tidyverse.org/>

tidyverse é uma coleção de pacotes R para ciência de dados que inclui dplyr para manipulação de dados e ggplot2 para visualização.

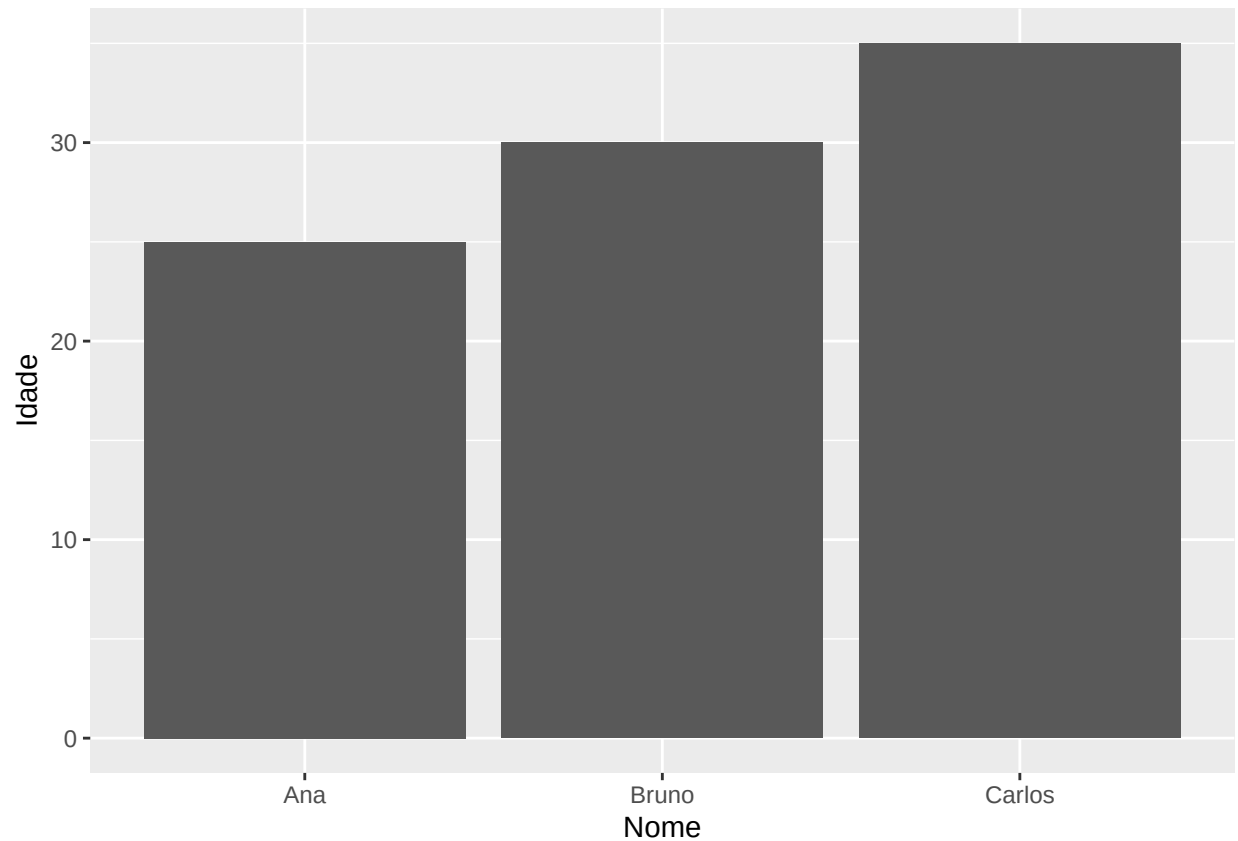
```
#install.packages("tidyverse")
```

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4.9000      v readr      2.1.5
## v forcats    1.0.0          v stringr    1.5.1
## v ggplot2     3.5.0          v tibble     3.2.1
## v lubridate  1.9.3          v tidyr      1.3.1
## v purrr      1.0.2
## -- Conflicts ----- tidyverse_conflicts() --
## x readr::edition_get() masks testthat::edition_get()
## x dplyr::filter()      masks stats::filter()
## x purrr::is_null()     masks testthat::is_null()
## x dplyr::lag()          masks stats::lag()
## x readr::local_edition() masks testthat::local_edition()
## x dplyr::matches()      masks tidyr::matches(), testthat::matches()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
# Exemplo com dplyr
data <- tibble(
  Nome = c('Ana', 'Bruno', 'Carlos'),
  Idade = c(25, 30, 35)
)

# Exemplo com ggplot2
ggplot(data, aes(x = Nome, y = Idade)) + geom_col()
```



Lubridate <https://lubridate.tidyverse.org/>

lubridate é um pacote em R para trabalhar de forma eficaz com datas e horas.

```
#install.packages("tidyverse")
```

```
library(lubridate)
```

```
# Exemplo de manipulação de datas
```

```
data_atual <- today()
```

```
print(data_atual)
```

```
## [1] "2024-03-23"
```

data.table O `data.table` é um pacote R para manipulação de dados de alta performance. É uma extensão do `data.frame` que permite operações de alto nível com grande eficiência e velocidade.

```
library(data.table)
```

```
##
```

```
## Attaching package: 'data.table'
```

```
## The following objects are masked from 'package:lubridate':
##
##     hour, isoweek, mday, minute, month, quarter, second, wday, week,
##     yday, year

## The following objects are masked from 'package:dplyr':
##
##     between, first, last

## The following object is masked from 'package:purrr':
##
##     transpose
```

```
# Criando um data.table
dt <- data.table(
  Nome = c('Ana', 'Bruno', 'Carlos'),
  Idade = c(25, 30, 35)
)

# Exemplo de manipulação de dados
dt[Nome == "Ana", Idade := 26] # Atualiza a idade de Ana
print(dt)
```

```
## Index: <Nome>
##      Nome Idade
##      <char> <num>
## 1:   Ana    26
## 2: Bruno    30
## 3: Carlos   35
```

shiny <https://shiny.rstudio.com/>

shiny é um pacote que permite criar aplicativos web interativos em R sem a necessidade de conhecimentos em HTML, CSS ou JavaScript.

```
#install.packages("shiny")
```

```
library(shiny)
```

```
# Exemplo básico de um app Shiny
ui <- fluidPage(
  titlePanel("Hello Shiny!"),
  sidebarLayout(
    sidebarPanel(
      sliderInput("num", "Escolha um número", 1, 100, 50),
      actionButton("goButton", "Enviar")
    ),
    mainPanel(
      textOutput("caption")
    )
  )
)
```

```
#ui
```

stringr <https://stringr.tidyverse.org/>

stringr é um pacote para manipulação de strings que torna fácil e consistente trabalhar com textos em R.

```
library(stringr)
```

```
texto <- "Ciência de Dados em R"
texto_maiusculo <- str_to_upper(texto)
print(texto_maiusculo)
```

```
## [1] "CIÊNCIA DE DADOS EM R"
```

purrr <https://purrr.tidyverse.org/>

purrr melhora a programação funcional em R, fornecendo um conjunto de ferramentas para trabalhar com funções e vetores.

```
library(purrr)
```

```
# Usando map para aplicar uma função a cada elemento de um vetor
resultados <- map(1:5, ~ .x * 2)
print(resultados)
```

```
## [[1]]
## [1] 2
##
## [[2]]
## [1] 4
##
## [[3]]
## [1] 6
##
## [[4]]
## [1] 8
##
## [[5]]
## [1] 10
```

leaflet <https://rstudio.github.io/leaflet/>

leaflet é um pacote para criar mapas interativos em R. É uma interface para a biblioteca JavaScript Leaflet.

```
#install.packages("leaflet")
```

```
library(leaflet)
```

```
# Exemplo de mapa interativo
mapa <- leaflet() %>% addTiles() %>% setView(lng = -47.92, lat = -15.78, zoom = 4)
mapa
```

Google Chrome was not found. Try setting the 'CHROMOTE_CHROME' environment variable to the executable



plotly <https://plotly-r.com/>

plotly permite a criação de gráficos interativos em R, baseados na biblioteca Plotly JavaScript.

```
#install.packages("plotly")
```

```
library(plotly)
```

```
##
## Attaching package: 'plotly'

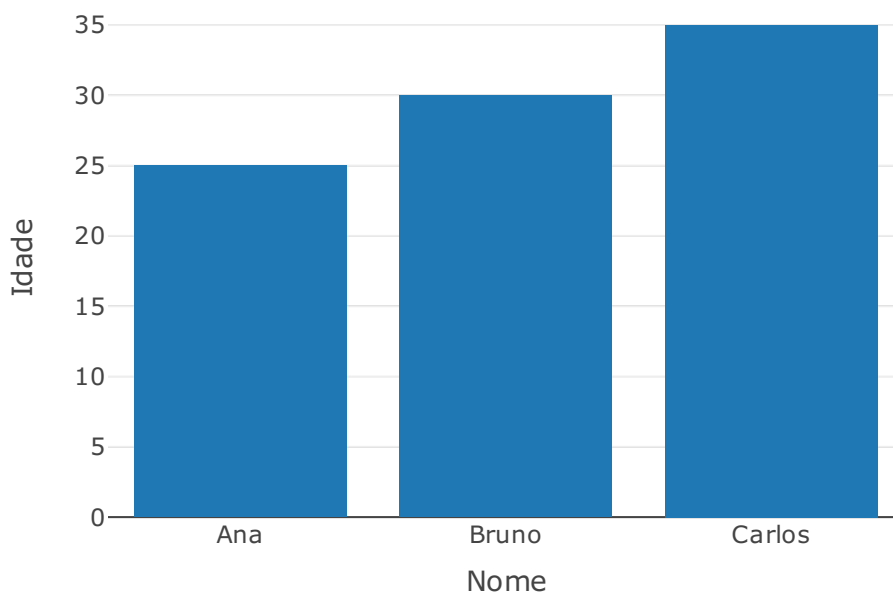
## The following object is masked from 'package:ggplot2':
##
##   last_plot

## The following object is masked from 'package:stats':
##
##   filter
```



```
## The following object is masked from 'package:graphics':  
##  
## layout
```

```
data <- data.table(  
  Nome = c('Ana', 'Bruno', 'Carlos'),  
  Idade = c(25, 30, 35)  
)  
  
# Exemplo de gráfico interativo  
fig <- plot_ly(data, x = ~Nome, y = ~Idade, type = 'bar')  
fig
```



Reticulate O pacote reticulate no R facilita a execução de código Python dentro do ambiente R, permitindo que integrem os recursos das duas linguagens em um único workflow. Isso é particularmente útil em situações onde uma tarefa específica é mais facilmente realizada ou tem bibliotecas mais desenvolvidas em Python do que em R.

```
#install.packages("reticulate")
```

```
library(reticulate)
```

```
#os <- import("os")
#os$listdir(".")

#!pip install pandas -q
#import pandas

#print(pandas.__version__)
```

Orientação a Objetos

R não é conhecido pela sua programação orientada a objetos (POO), mas ela suporta este paradigma. usando S3 e S4, além do sistema R6, que é mais parecido com a POO em linguagens como Python.

S3

- **Classes S3:** O sistema S3 é um informal e baseado em convenções.
- **Métodos S3:** Os métodos são funções que têm um argumento especial (geralmente o primeiro) que determina a classe do objeto.

```
# Definindo uma classe S3
carro <- list(marca = "Toyota", modelo = "Corolla")
class(carro) <- "Carro"

# Definindo um método para a classe
print.Carro <- function(x) {
  cat("Marca:", x$marca, "- Modelo:", x$modelo, "\n")
}

# Testando o método
print(carro)
```

```
## Marca: Toyota - Modelo: Corolla
```

S4

- **Classes e Métodos S4:** O sistema S4 é mais formal e robusto, permitindo a definição de classes com validação de tipos e métodos específicos para essas classes.

```
# Definindo uma classe S4
setClass("Veiculo",
  slots = list(marca = "character", modelo = "character"))

# Criando um método S4
setMethod("show", "Veiculo", function(object) {
  cat("Marca:", object@marca, "- Modelo:", object@modelo, "\n")
})

# Criando uma instância e testando
veiculo <- new("Veiculo", marca = "Ford", modelo = "Fiesta")
show(veiculo)
```

```
## Marca: Ford - Modelo: Fiesta
```

R6

- **R6:** O sistema R6 é mais semelhante à POO em linguagens como Python. Ele permite métodos e propriedades de objetos, e é mais adequado para modelagem OO avançada.

```
# Carregando o pacote R6
library(R6)

# Definindo uma classe R6
CarroR6 <- R6Class("CarroR6",
  public = list(
    marca = NULL,
    modelo = NULL,
    initialize = function(marca, modelo) {
      self$marca <- marca
      self$modelo <- modelo
    },
    exibir_info = function() {
      cat("Marca:", self$marca, "- Modelo:", self$modelo, "\n")
    }
  )
)

# Criando um objeto e chamando um método
meu_carro <- CarroR6$new("Toyota", "Corolla")
meu_carro$exibir_info()
```

```
## Marca: Toyota - Modelo: Corolla
```

Refatoração

Refatoração é o processo de reestruturar o código existente em R sem alterar seu comportamento externo. Seu objetivo é melhorar a clareza, eficiência e facilidade de manutenção do código. **Princípios da Refatoração em R:**

- **Melhorar a Legibilidade:** Tornar o código mais claro e compreensível.
- **Reduzir Complexidade:** Simplificar funções e estruturas complicadas.
- **Eliminar Código Redundante:** Remover repetições e utilizar funções eficientes.
- **Otimizar Desempenho:** Usar estruturas de dados e funções nativas para melhor eficiência.
- **Corrigir Problemas Latentes:** Resolver problemas potenciais não óbvios. Suponha que temos uma função em R que calcula o total de vendas com base em vetores de preços e quantidades. O código original pode ser algo assim:

```
calcular_total_vendas <- function(preços, quantidades) {
  total <- 0
  for (i in seq_along(preços)) {
    total <- total + preços[i] * quantidades[i]
  }
  total
}
```

```
preços <- c(100, 200, 300)
quantidades <- c(1, 2, 3)
print(calcular_total_vendas(preços, quantidades))
```

```
## [1] 1400
```

Podemos refatorar este código para torná-lo mais eficiente e legível. Por exemplo, podemos usar a função `mapply` para aplicar uma função de forma mais direta.

```
calcular_total_vendas_refatorado <- function(preços, quantidades) {
  sum(mapply('*', preços, quantidades))
}

print(calcular_total_vendas_refatorado(preços, quantidades))
```

```
## [1] 1400
```

Na versão refatorada, utilizamos `mapply` para multiplicar cada par de preço e quantidade e `sum` para calcular o total. Isso torna o código mais conciso, legível e potencialmente mais eficiente.

Referência Complementar

Wickham, H.; **Advanced R**. Chapman & Hall/CRC, 2015